

An Empirical Study on the Efficacy of ESLint in Reducing Bugs in TypeScript

Literature Review - CMPT 479/982 - Fall 2025 - Professor Saba Alimadadi

Sara Magdalinski (sma274@sfu.ca) & Nicholas Tait (nta52@sfu.ca)

I. INTRODUCTION AND BACKGROUND

Static analysis tools, such as linters, are widely used in modern software development to identify potential coding issues, enforce best practices, and improve code readability. In the JavaScript and TypeScript ecosystems, ESLint has emerged as the most popular linter [1], [2].

However, despite its popularity and widespread adoption, there is limited quantitative evidence regarding whether using a linter actually leads to fewer bugs in real-world projects. Much of the existing research provides valuable insights into human factors surrounding linters, investigating why and how developers adopt tools like ESLint, which rules they choose to enable or disable, and the various barriers they encounter, such as configuration effort and managing false positives [1], [2], [3], [4], [5], [6]. Other studies have explored the effectiveness of static analysis tools in different ecosystems like Java, using methodologies such as pre-fix snapshot analysis or correlating warning density with defects [7], [8], [9], [10]. Within the JavaScript and TypeScript context itself, research has confirmed the quality benefits of TypeScript's static typing over plain JavaScript and shown that tooling choices correlate with certain project outcomes, but these studies often stop short of isolating the specific contribution of linters like ESLint to reducing bug frequency [8], [11], [12].

Therefore, there is currently limited quantitative evidence directly addressing whether the adoption and configuration strictness of ESLint correlate with a measurable reduction in bugs in real-world TypeScript projects. Our project aims to fill this critical gap by empirically studying the relationship between ESLint usage patterns (specifically, its adoption and the strictness of its configuration) and bug-fix activity across a diverse set of open-source TypeScript repositories hosted on GitHub. We seek to determine if projects utilizing ESLint exhibit lower bug-fix ratios, if stricter linting configurations are associated with improved software quality indicators, and ultimately, to estimate the fraction of real-world bugs that might be considered linter-preventable.

To guide our investigation, we attempt to answer the following research questions:

RQ1: How prevalent is the adoption of ESLint among open-source TypeScript Projects, and how strict are their configurations?

RQ2: Do TypeScript projects that use ESLint exhibit lower bug-fix ratios compared to those that do not?

RQ3: Is there a relationship between the strictness of a project's ESLint configuration and its bug-fix activity?

RQ4: For a subset of bug-fix commits, what fraction are linter-preventable under (a) the project's actual config (if available) and (b) a standard TypeScript ESLint config?

II. REVIEW OF RELATED WORK

We categorize the related research into three key areas to situate our project within the existing literature, understand the state of the art, and identify the gaps our research will address. These categories are: Developer Adoption and Usage of Static Analysis Tools, Effectiveness of Static Analysis for Defect Detection, and Empirical Studies on JavaScript/TypeScript Quality and Tooling.

A. Developer Adoption and Usage of Static Analysis

A significant body of research investigates the human factors of static analysis, exploring the complex relationship between developers and the tools designed to assist them. This work reveals that while static analysis tools are widely available and often adopted, their effective usage is significantly affected by developer experience, project context, and practical trade-offs. The literature shows a clear gap between a tool's theoretical bug-finding potential and its real-world application.

[1] and [2] conducted extensive case studies on ESLint. Through developer interviews and surveys, they found that developers primarily adopt linters to maintain code consistency, prevent errors, augment unit tests, automate style enforcement, and save discussion time. However, this adoption is balanced by significant practical challenges. The

most common pain points include the difficulty of agreeing on rules as a team, the effort of creating and maintaining configurations, dealing with false positives, and the social and technical friction of enforcing rules. Some developers choose not to use linters at all, finding that they can complicate the build process and increase the cost of the development life cycle. To manage these costs, developers adopt various configuration strategies, such as using an already existing preset, choosing rules that fit the current project, or opting for a minimal configuration.

[3] conducted surveys and interviews to understand how developers engage with Automatic Static Analysis Tools (ASATs) in different contexts, such as local programming, code review, and continuous integration. They found that although most developers (78%) use the same ASAT configuration across these contexts, a significant majority (71%) pay attention to different types of warnings depending on the specific task. This study highlights that while developers see the potential of ASATs, challenges like difficult configuration and perceived tool unreliability limit their practical enforcement, especially for non-style-related checks.

[4] surveyed and interviewed Microsoft developers to understand their needs regarding program analysis tools. They found the largest barrier to adoption is having irrelevant checks enabled by default. Developers also struggle with bad warning messages, too many false positives, and tools being too slow or difficult to fit into the workflow. They discovered an almost even split when comparing the trade-off of reporting false positives versus missing real issues, with 49.3% of developers preferring fewer false positives even if some real issues may be missed.

[5] investigated why developers underutilize static analysis tools through developer interviews and discovered similar results to the prior mentioned studies. They found that while developers recognize the benefit of automating bug detection to save time and effort, significant barriers hinder adoption. These include the high cognitive load from false positives and large volumes of warnings, poorly presented or difficult to understand warning messages that lack context or actionable advice, difficulty configuring tools or managing rulesets within a team, and

inadequate integration into the developers' workflow, often being perceived as slow or disruptive.

Adding to these challenges, [6] specifically examined the quality and understandability of bug reports generated by the Clang Static Analyzer. Surveying reports from several open-source projects, they found a substantial fraction (11 out of 31) were unacceptable, being either incomprehensible or lacking sufficient information for a developer to make a judgment.

[10] analyzed commit histories of JavaScript packages and observed that linter choices and configurations are rarely changed once introduced, and when modifications do occur, they are typically minor. [13] conducted a large-scale study analyzing the state of static analysis across over 168,000 open-source projects and found that while ASAT usage is common, configurations rarely deviate from the defaults, and developers rarely add custom rules. Similar to [10], their analysis showed that over 80% of ASAT configuration files remain unchanged after their initial creation, with any modifications typically being minor and occurring very early in the project's history.

Finally, [14] studied lint-based warnings in Python projects, finding they are quite frequent (present in 39% of projects). Their survey revealed that developers prefer code without these warnings, and this preference increases with experience. Furthermore, pull requests aimed at fixing these warnings had a high acceptance rate (71.8%), suggesting developers value these corrections when they are pointed out.

1) Connections and Comparisons: The reviewed studies present a consistent narrative of developer engagement with static analysis tools across different programming ecosystems. A clear trend shows that developers have similar motivations for adoption, primarily to enforce code consistency and prevent errors, while also facing a common set of significant barriers. These include the high cognitive load from false positives, the effort of configuration and maintenance, poor workflow integration, and the difficulty of understanding warnings. In response, developers employ strategies such as using presets or minimal configurations, which [10] and [13] observe tend to remain stable over time.

Despite these challenges, the high acceptance rate of pull requests fixing lint warnings, found by [14], suggests an underlying appreciation for the value of these rules. The primary gap across these studies is their focus on developer behaviour and perception rather than the quantifiable impact of linter usage on software quality metrics like bug rates.

2) *Relevance to Project*: The existing literature surrounding developer adoption of static analysis tools directly motivates and complements our proposed study. It validates the premise of our first research question (RQ1) by confirming that linter adoption is prevalent but that configuration practices are highly variable, making strictness a meaningful metric to investigate.

Furthermore, the consistent identification of significant barriers, such as managing false positives, the effort of configuration, workflow disruption, and poor warning understandability, explains that this variability exists because developers actively make trade-offs leading to different levels of tool adoption and configuration strictness [1], [2], [3], [4], [5], [6]. This directly motivates our investigation in RQ2 and RQ3, which asks whether these different strategies and resulting levels of strictness actually correlate with different bug-fix frequencies. The findings that configurations tend to be stable over time also make a correlational study like ours more feasible, as we can examine the association between a relatively consistent configuration state and project outcomes [10], [13].

Finally, the evidence suggesting developers value and act on linter warnings when presented appropriately reinforces the importance of RQ4, quantifying the potential preventive impact of these tools against real-world bugs [14].

B. Effectiveness of Static Analysis for Defect Detection

This category of research moves beyond perception to quantify the effectiveness of static analysis tools in detecting actual defects, primarily within the Java ecosystem. These studies employ different methodologies but converge on the findings that while tools can detect real bugs, their

effectiveness is often limited, and the specified rules or warnings matter.

[7] attempts to determine if static analysis tools help prevent actual defects that occur in the field and those reported by users. Additionally, they ask if these tools had been used, would they have detected these defects and warned the programmer to fix them? To answer these questions, the authors perform an empirical study on state-of-the-art static bug finding tools on hundreds of reported and fixed defects extracted from three open source programs. They found that many defects could be detected, but a substantial portion could not. They also analyze the types of tool warnings that are more effective in finding field defects and characterize the types of missed defects. To extract the faulty lines of code, the authors employ a method to extract the version of the program in the repository prior to the fix and then download the version after the fix, using Git diff to determine what lines were the root cause of the issue.

[9] investigated the project-level correlation between PMD warning density and external software quality. They analyzed project histories, comparing warning density in bug-inducing files versus the rest of the project, controlling for time trends. They also examine the impact of different rule sets. They found that while bug-inducing changes introduced slightly more warnings, the overall relationship between warning density and defects was statistically significant but had a negligible effect size. However, stronger signals were observed when restricting analysis to a curated default rule set.

[15] focused on precision (true positives among warnings) by asking what percentage and type of FindBugs issues correspond to actual defects in student-written Java projects. An issue was considered related to a defect if (a) the issue disappeared between the lab and home versions, (b) a test case went from failing to passing between these versions, and (c) the source code line flagged by the issue was modified during the fix. They found that only a limited set of FindBugs issues exhibited high precision, while many others had low precision. They suggest that developers could prioritize or enable only the high-precision rules to reduce information load.

1) *Connections and Comparisons*: All three studies empirically evaluate the effectiveness of Java static analysis tools in relation to real defects. They collectively challenge the assumption that simply running a tool guarantees significant quality improvement.

These studies operate at different levels. [7] and [15] focus on the micro-level, linking specific warnings to specific lines of code involving a fix. [9] focused on the macro-level, correlating aggregated warning density across files/projects with bug frequencies. Furthermore, [7] measures recall (false negatives), while [15] measures precision (fraction of warnings corresponding to actual defects). [9] measures statistical correlation.

Despite the differences, several trends emerge. Static analysis tools miss many real bugs [7] and many warnings do not correspond to fixed defects [15], leading to weak overall correlation [9]. The specific type or set of rules matters. Effectiveness varies by warning family [7], and only a few rules show high precision [15]. Additionally, curated rule sets yield slightly stronger correlation [9]. Moreover, context matters. [15] notes differences might arise based on context, while [9] highlights the importance of accounting for time trends in warning density.

2) *Relevance to Project*: The work by [7] provides a direct methodological foundation for our RQ4 (Linter preventable deep-dive). We adapt their approach to estimate the recall for ESLint rules in TypeScript. [15]’s work complements this by highlighting the importance of precision and identifying specific high-value rules, supporting our plan in RQ4 to categorize types of preventable bugs identified by ESLint rules.

[9] serves as a key precedent for our project-level analysis in RQ2 and RQ3. Their findings of negligible effects of PMD in Java motivate our investigation into whether ESLint shows a stronger correlation in TypeScript. Their emphasis on curated rules sets directly supports our RQ3’s focus on strictness, where we examine if enabling more “error” rules correlates with lower bug-fix ratios.

C. Empirical Studies on JavaScript/TypeScript Quality and Tooling

This category of research focuses specifically on the JavaScript and TypeScript ecosystems, empirically investigating how language choices and tooling impact software quality. A central theme is the effect of static typing introduced by TypeScript.

[8] empirically investigated the effectiveness of static type systems (Flow and TypeScript) in detecting bugs within JavaScript projects. Using a novel approach, they identified bug-fixing commits, checked out the code before the fix, and manually added type annotations consistent with the intended behaviour revealed by the fix. By running Flow and TypeScript on this annotated buggy code, they quantified that both type systems could have detected approximately 15% of these public, real-world bugs, providing strong evidence for the quality benefits of static typing in the ecosystem.

[11] provided further evidence through a systematic comparison of GitHub projects, concluding that TypeScript applications exhibited significantly better code quality and code understandability compared to JavaScript counterparts. However, they surprisingly found that TypeScript projects in their sample had a higher bug fix commit ratio and slightly longer bug resolution times.

Beyond the language choice, the specific tooling used within the ecosystem also impacts development. [12] empirically demonstrated that the choice of quality assurance tools (including linters) impacts project outcomes like code churn and issue prevalence in JavaScript projects. Counterintuitively, they found that while developers do not engage in much discussion surrounding which tool to adopt, the choice of tool matters significantly, especially in the cases of ESLint and coveralls. This finding directly supports the premise that specific tooling decisions within the ecosystem can have measurable effects on development activity and quality indicators.

The findings of [10] fit into this section as well. In their study of JavaScript projects, they found that while the number of linter warnings tends to increase with project size, the warning density (warnings per KLOC) shows a slight decreasing trend over time, suggesting a potential improvement in

code quality relative to the project’s growth. This offers an insight into the long-term impact of tooling.

[16] highlighted the inherent limitations of static analysis by introducing DLint, a dynamic linter. They found that DLint detected an average of 49 problems per site that were missed by the static linter JSHint. This study helps define the scope of errors that static analysis can realistically prevent.

1) Connections and Comparisons: The studies in this category collectively demonstrate an ongoing empirical effort to quantify how language features and ecosystem tooling influence software quality within the JavaScript and TypeScript domain. Across [8] and [11], results consistently indicate that the introduction of static typing enhances code readability and structural quality, yet its direct impact on reducing real-world bugs remains unclear.

The focus on tooling by [10] and [12] complements these language-level comparisons. [12] shows that the choice of a tool has measurable correlation with project outcomes, while [10] demonstrates that the long-term usage of these tools correlates with a decreasing density of warnings. These studies differ in their metrics, ranging from potential bug prevention to bug fix ratios, code churn, and warning density, making a direct comparison of tool effectiveness challenging. Providing an opposing perspective to static analysis tools, [16] highlights the methodological limitations showing that static analysis misses a significant number of runtime issues discoverable through dynamic analysis.

The primary gap that emerges is the absence of studies directly measuring the effectiveness of specific tooling configurations, like linters, on actual bug rates. For instance, [11] examined bug ratios but focused on the overall JavaScript versus TypeScript comparison, not the specific effect of linting within TypeScript. Similarly, [12] linked tool choice to process metrics like churn, not bug rates, contingent on configuration strictness, and [10] used warning density to measure quality rather than measuring actual bugs. Our research aims to fill this gap by directly measuring the relationship between the strictness of ESLint configurations and project-level bug-fix frequency within TypeScript projects.

2) Relevance to Project: The existing literature within this category provides the direct ecosystem context

for our proposed research. The findings that TypeScript improves certain quality aspects over JavaScript from [8] and [11] establish a baseline, allowing our study to investigate the additional benefit that ESLint might offer.

Our project builds upon and differs significantly from this prior work. While [12] showed that tool choices impact project outcomes like code churn and issue prevalence, our study focuses specifically on bug-fix ratios as a more direct measure of quality, forming the core of RQ2 and RQ3. This directly addresses the research gap concerning the effectiveness of linters in reducing bugs, a question left open by [11]’s surprising finding that TypeScript projects had a higher bug-fix commit ratio than JavaScript. Furthermore, our approach in RQ4, using pre-fix snapshot analysis adapted from [7] (discussed in Section B) and [8], complements these studies by attempting to quantify the preventive impact of ESLint. This is especially relevant in the context of [16]’s work, which highlights the inherent limitations of static analysis. Our study aims to measure the effectiveness of ESLint within those known limitations.

III. POTENTIAL IMPACT

The potential impact of this research could affect many groups.

For developers and teams, our findings could provide concrete evidence on whether investing effort into configuring and enforcing stricter ESLint rules translates into tangible quality improvements (fewer bugs). This could guide the decision on tooling adoption and configuration strategies within TypeScript projects.

By identifying what rules are most frequently associated with preventable bugs (RQ4), our study could inform the prioritization of rule development, documentation and default configuration for tool maintainers.

For the research community, this study contributes empirical data to the ongoing debate about the effectiveness of static analysis tools, specifically within the growing TypeScript ecosystem. It addresses the gap highlighted by [11] regarding TypeScript’s impact on bug rates and provides a method for similar studies on other linters or languages. Furthermore, our curated dataset will enable replication and extension studies.

IV. SUMMARY

This review reveals a clear gap in existing research. While studies have examined why developers use linters and others have measured the effectiveness of type systems or compared different languages, a link between them all is missing. The findings from [11] that TypeScript adoption alone may not reduce bug ratios make our investigations important. No major study has yet performed a large-scale, quantitative analysis to determine if the adoption and strict configuration of ESLint correlates with a lower bug-fix ratio, specifically in the TypeScript environment. Our project directly addresses this need.

Additionally, our project addresses three distinct but related areas of research: developer adoption of static analysis, the effectiveness of static analysis for defect detection, and empirical studies within the JavaScript / TypeScript ecosystem. We build upon studies of developer behaviour, linter adoption and defect analysis methodologies to provide an empirical study on whether ESLint is effective at reducing bugs in real-world TypeScript projects.

REFERENCES

- [1] K. F. Tómasdóttir, M. Aniche, and A. Van Deursen, “The Adoption of JavaScript Linters in Practice: A Case Study on ESLint,” *IEEE Transactions on Software Engineering*, vol. 46, no. 8, pp. 863–891, Aug. 2020, doi: [10.1109/TSE.2018.2871058](https://doi.org/10.1109/TSE.2018.2871058).
- [2] K. F. Tómasdóttir, M. Aniche, and A. van Deursen, “Why and how JavaScript developers use linters,” in *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*, Oct. 2017, pp. 578–589, doi: [10.1109/ASE.2017.8115668](https://doi.org/10.1109/ASE.2017.8115668).
- [3] C. Vassallo, S. Panichella, F. Palomba, S. Proksch, H. C. Gall, and A. Zaidman, “How developers engage with static analysis tools in different contexts,” *Empir Software Eng*, vol. 25, no. 2, pp. 1419–1457, Mar. 2020, doi: [10.1007/s10664-019-09750-5](https://doi.org/10.1007/s10664-019-09750-5).
- [4] M. Christakis and C. Bird, “What developers want and need from program analysis: an empirical study,” in *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering*, Singapore Singapore: ACM, Aug. 2016, pp. 332–343, doi: [10.1145/2970276.2970347](https://doi.org/10.1145/2970276.2970347).
- [5] B. Johnson, Y. Song, E. Murphy-Hill, and R. Bowdidge, “Why don’t software developers use static analysis tools to find bugs?,” in *2013 35th International Conference on Software Engineering (ICSE)*, May 2013, pp. 672–681, doi: [10.1109/ICSE.2013.6606613](https://doi.org/10.1109/ICSE.2013.6606613).
- [6] K. Umann and Z. Porkoláb, “A Case Study on the Quality of Static Analysis Bug Reports”.
- [7] F. Thung, Lucia, D. Lo, L. Jiang, F. Rahman, and P. T. Devanbu, “To what extent could we detect field defects? an empirical study of false negatives in static bug finding tools,” in *Proceedings of the 27th IEEE/ACM International Conference on Automated Software Engineering*, Essen Germany: ACM, Sept. 2012, pp. 50–59, doi: [10.1145/2351676.2351685](https://doi.org/10.1145/2351676.2351685).
- [8] Z. Gao, C. Bird, and E. T. Barr, “To Type or Not to Type: Quantifying Detectable Bugs in JavaScript,” in *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*, Buenos Aires: IEEE, May 2017, pp. 758–769, doi: [10.1109/ICSE.2017.75](https://doi.org/10.1109/ICSE.2017.75).
- [9] A. Trautsch, S. Herbold, and J. Grabowski, “Are automated static analysis tools worth it? An investigation into relative warning density and external software quality on the example of Apache open source projects,” *Empir Software Eng*, vol. 28, no. 3, p. 66, Apr. 2023, doi: [10.1007/s10664-023-10301-2](https://doi.org/10.1007/s10664-023-10301-2).
- [10] T. Heričko and B. Šumak, “Analyzing Linter Usage and Warnings Through Mining Software Repositories: A Longitudinal Case Study of JavaScript Packages,” in *2022 45th Jubilee International Convention on Information, Communication and Electronic Technology (MIPRO)*, May 2022, pp. 1375–1380, doi: [10.23919/MIPRO55190.2022.9803554](https://doi.org/10.23919/MIPRO55190.2022.9803554).
- [11] J. Bogner and M. Merkel, “To Type or Not to Type? A Systematic Comparison of the Software Quality of JavaScript and TypeScript Applications on GitHub,” in *Proceedings of the 19th International Conference on Mining Software Repositories*, May 2022, pp. 658–669, doi: [10.1145/3524842.3528454](https://doi.org/10.1145/3524842.3528454).
- [12] D. Kavalier, A. Trockman, B. Vasilescu, and V. Filkov, “Tool Choice Matters: JavaScript Quality Assurance Tools and Usage Outcomes in GitHub Projects,” in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, May 2019, pp. 476–487, doi: [10.1109/ICSE.2019.00060](https://doi.org/10.1109/ICSE.2019.00060).
- [13] M. Beller, R. Bholanath, S. McIntosh, and A. Zaidman, “Analyzing the State of Static Analysis: A Large-Scale Evaluation in Open Source Software,” in *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering*

(SANER), Mar. 2016, pp. 470–481. doi: [10.1109/SANER.2016.105](https://doi.org/10.1109/SANER.2016.105).

[14] N. Oliveira, M. Ribeiro, R. Bonifácio, R. Gheyi, I. Wiese, and B. Fonseca, “Lint-Based Warnings in Python Code: Frequency, Awareness and Refactoring,” in *2022 IEEE 22nd International Working Conference on Source Code Analysis and Manipulation (SCAM)*, Oct. 2022, pp. 208–218. doi: [10.1109/SCAM55253.2022.00030](https://doi.org/10.1109/SCAM55253.2022.00030).

[15] A. Vetro, M. Morisio, and M. Torchiano, “An empirical validation of FindBugs issues related to defects,” in *15th Annual Conference on Evaluation & Assessment in Software Engineering (EASE 2011)*, Apr. 2011, pp. 144–153. doi: [10.1049/ic.2011.0018](https://doi.org/10.1049/ic.2011.0018).

[16] L. Gong, M. Pradel, M. Sridharan, and K. Sen, “DLint: dynamically checking bad coding practices in JavaScript,” in *Proceedings of the 2015 International Symposium on Software Testing and Analysis*, Baltimore MD USA: ACM, July 2015, pp. 94–105. doi: [10.1145/2771783.2771809](https://doi.org/10.1145/2771783.2771809).